Python Niveau I — Exercices Jour 1

Module 1 : Premiers pas avec Python

Exercice 1.1 : La console comme calculatrice [Difficulté : Facile]

Énoncé : Utilisez la console Python pour calculer les expressions suivantes :

1. 45 + 32
2. 256 / 8
3. 2 puissance 10
4. Le reste de la division de 17 par 5
5. La division entiere de 17 par 5

Indice : L'operateur puissance en Python est `**`. Le modulo se note `%` et la division entiere `//`.

▶ Solution

Exercice 1.2 : Créer et manipuler des variables [Difficulté : Facile]

Énoncé :

1. Creez une variable `prix_ht` contenant la valeur 85.50
2. Creez une variable `tva` contenant 0.20
3. Calculez le prix TTC et stockez-le dans `prix_ttc`
4. Affichez le prix TTC avec un f-string : "Le prix TTC est de X euros"
5. Verifiez le type de chaque variable avec `type()`

Indice : L'assignation en Python utilise `=`. Les f-strings s'ecrivent `f"texte {variable}"`. Le prix TTC = prix HT x (1 + TVA).

▶ Solution

Exercice 1.3 : Écrire un script complet [Difficulté : Facile]

Énoncé : Ecrivez un script Python qui :

1. Definit le rayon d'un cercle (`rayon = 7`)
2. Calcule le perimetre (2 x pi x r)
3. Calcule l'aire (pi x r^2)
4. Affiche les deux resultats avec des f-strings formates a 2 decimales

Indice : Importez pi depuis le module `math` avec `from math import pi`. Pour formater a 2 decimales dans un f-string : `f"{valeur:.2f}"`.

▶ Solution

Exercice 1.4 : Utiliser l'aide [Difficulté : Facile]

Énoncé :

1. Consultez l'aide de la fonction `round()` avec `help(round)`
2. Arrondissez la valeur 3.14159265 a 2 decimales
3. Arrondissez la meme valeur a 4 decimales
4. Utilisez `type()` sur differentes valeurs : 42, 3.14, "bonjour", True
5. Utilisez `dir()` sur une chaine de caracteres pour decouvrir ses methodes

Indice : `help(fonction)` affiche l'aide. `dir(objet)` liste les attributs et methodes disponibles. `type(objet)` retourne le type.

▶ Solution

Exercice 1.5 : Environnement de travail [Difficulté : Intermédiaire]

Énoncé :

1. Importez le module `sys` et affichez la version de Python avec `sys.version`
2. Importez le module `math` et affichez la valeur de `math.pi` et `math.e`
3. Utilisez `math.sqrt(144)` et `math.factorial(6)`
4. Importez `datetime` et affichez la date du jour

5. Listez les fonctions disponibles dans le module `math` (sans les fonctions privees)

Indice : `import module` charge un module. `sys.version` contient la version Python. `datetime.date.today()` donne la date du jour.

▶ Solution

Module 2 : Structures de données

Exercice 2.1 : Types fondamentaux [Difficulté : Facile]

Énoncé :

1. Creez une variable de chaque type : `int`, `float`, `str`, `bool`
2. Verifiez le type de chaque variable avec `type()`
3. Testez `isinstance()` pour verifier si une variable est d'un type donne
4. Effectuez des conversions : `int("42")`, `float("3.14")`, `str(100)`, `bool(0)`, `bool(1)`, `bool("")`, `bool("texte")`

Indice : `isinstance(variable, type)` retourne `True` ou `False`. Les valeurs "falsy" en Python sont : `0`, `0.0`, `""`, `None`, `[]`, `{}`.

▶ Solution

Exercice 2.2 : Création de listes [Difficulté : Facile]

Énoncé : Creez les listes suivantes :

1. `l1` : les nombres de 1 a 20 avec `list(range(...))`
2. `l2` : les nombres pairs de 2 a 30
3. `l3` : une liste de 5 zeros avec `[0] * 5`
4. `l4` : les carres de 1 a 10 avec une comprehension de liste
5. `l5` : les valeurs `[4, 8, 15, 16, 23, 42]`

Affichez la longueur de chaque liste.

Indice : `range(debut, fin, pas)` genere une sequence (fin exclu). `list()` convertit en liste. Les comprehensions s'ecrivent `[expression for x in iterable]`.

▶ Solution

Exercice 2.3 : Indexation et slicing [Difficulté : Intermédiaire]

Énoncé : Soit la liste `notes = [12, 15, 8, 17, 14, 9, 11, 16, 7, 13]`.

1. Extrayez la 3e note (index 2)
2. Extrayez la derniere note avec un index negatif
3. Extrayez les notes de l'index 1 a 4 (slice)
4. Extrayez un element sur deux avec le pas (step)
5. Inversez la liste avec le slicing `[::-1]`
6. Extrayez les 3 derniers elements

Indice : L'indexation Python commence a 0. Le slicing s'ecrit `[debut:fin:pas]` ou `fin` est exclu. Les indices negatifs comptent depuis la fin.

▶ Solution

Exercice 2.4 : Opérations sur listes [Difficulté : Intermédiaire]

Énoncé : Soit `fruits = ["pomme", "banane", "orange"]`.

1. Ajoutez "fraise" a la fin avec `append()`
2. Inserez "kiwi" en position 1 avec `insert()`
3. Ajoutez `["mangue", "ananas"]` avec `extend()`
4. Supprimez "banane" avec `remove()`
5. Triez la liste par ordre alphabetique avec `sort()`
6. Inversez l'ordre avec `reverse()`
7. Affichez la liste apres chaque operation

Indice : `append()` ajoute a la fin, `insert(position, element)` insere, `extend()` ajoute plusieurs elements, `remove()` supprime par valeur, `sort()` trie en place, `reverse()` inverse en place.

▶ Solution

Exercice 2.5 : Tuples [Difficulté : Intermédiaire]

Énoncé :

1. Creez un tuple `coordonnees = (48.8566, 2.3522)` (latitude, longitude de Paris)
2. Accedez a la latitude et la longitude par indexation
3. Utilisez le unpacking : `lat, lon = coordonnees`
4. Creez un tuple de tuples representant 3 villes avec leurs coordonnees
5. Montrez qu'un tuple est immutable (essayez de modifier un element)
6. Utilisez un tuple comme cle de dictionnaire

Indice : Les tuples sont immutables : on ne peut pas modifier leurs elements. C'est pour cela qu'on peut les utiliser comme cles de dictionnaire (contrairement aux listes).

▶ Solution

Exercice 2.6 : Dictionnaires [Difficulté : Intermédiaire]

Énoncé : Creez un dictionnaire `etudiant` contenant :

- `nom` : "Dupont"
- `prenom` : "Marie"
- `age` : 22
- `notes` : [14, 16, 12, 18, 15]

Puis :

1. Accedez au prenom avec `[]` et avec `.get()`
2. Ajoutez une cle `email` avec la valeur "marie.dupont@univ.fr"
3. Affichez toutes les cles avec `.keys()`
4. Affichez toutes les valeurs avec `.values()`
5. Parcourez le dictionnaire avec `.items()`
6. Utilisez `.get()` avec une valeur par defaut pour une cle inexistante

Indice : `dict[cle]` leve une erreur si la cle n'existe pas, tandis que `dict.get(cle, defaut)` retourne la valeur par defaut.

▶ Solution

Exercice 2.7 : Ensembles (sets) [Difficulté : Intermédiaire]

Énoncé :

1. Creez deux ensembles : `a = {1, 2, 3, 4, 5}` et `b = {4, 5, 6, 7, 8}`
2. Calculez l'union de `a` et `b`
3. Calculez l'intersection
4. Calculez la difference `a - b` et `b - a`
5. Calculez la difference symetrique
6. Verifiez si 3 est dans `a` (test d'appartenance)
7. Creez un ensemble a partir d'une liste avec doublons et observez la deduplication

Indice : `|` pour l'union, `&` pour l'intersection, `-` pour la difference, `^` pour la difference symetrique. `in` pour l'appartenance.

▶ Solution

Exercice 2.8 : Conversions et mutabilité [Difficulté : Intermédiaire]

Énoncé :

1. Creez une liste, un tuple et un set contenant les memes elements `[1, 2, 3, 4, 5]`
2. Convertissez la liste en tuple, le tuple en set, le set en liste
3. Montrez que la liste est mutable (modifiez un element)
4. Montrez que le tuple est immutable (essayez de modifier un element)
5. Montrez qu'un set ne garde pas l'ordre et n'accepte pas les doublons
6. Creez une copie superficielle d'une liste et montrez la difference avec `=`

Indice : `list()`, `tuple()`, `set()` permettent les conversions. Pour copier une liste : `liste.copy()` ou `liste[:]`. L'assignation `b = a` ne copie pas, elle cree une

reference.

▶ Solution

Module 3 : Fonctions et contrôle

Exercice 3.1 : Conditions if/elif/else [Difficulté : Facile]

Énoncé :

1. Ecrivez un programme qui demande un age (utilisez une variable `age = 25`) et affiche :
    - "Mineur" si age < 18
    - "Jeune adulte" si 18 <= age < 30
    - "Adulte" si 30 <= age < 60
    - "Senior" si age >= 60
2. Ecrivez un programme qui categorise une note (variable `note = 14`) :
    - "Excellent" si >= 16
    - "Bien" si >= 14
    - "Assez bien" si >= 12
    - "Passable" si >= 10
    - "Insuffisant" si < 10

Indice : La syntaxe est `if condition:`, `elif condition:`, `else:`. L'indentation (4 espaces) est obligatoire en Python.

▶ Solution

Exercice 3.2 : Boucles for [Difficulté : Facile]

Énoncé :

1. Affichez les nombres de 1 a 10 avec `range()`
2. Parcourez la liste `fruits = ["pomme", "banane", "orange", "fraise"]` et affichez chaque fruit

3. Utilisez `enumerate()` pour afficher l'index et le fruit
4. Calculez la somme des nombres de 1 a 100 avec une boucle `for`
5. Parcourez un dictionnaire `{"a": 1, "b": 2, "c": 3}` avec `.items()`

Indice : `range(1, 11)` genere 1 a 10. `enumerate(liste)` retourne des paires `(index, element)`.

▶ Solution

Exercice 3.3 : Boucles while [Difficulté : Intermédiaire]

Énoncé :

1. Ecrivez un compte a rebours de 10 a 0 avec `while`
2. Calculez la factorielle de 7 avec une boucle `while`
3. Trouvez le plus petit entier `n` tel que `2**n > 1000`
4. Simulez un lancer de de : generez des nombres aleatoires entre 1 et 6 jusqu'a obtenir un 6 (utilisez `random.randint`)

Indice : La syntaxe est `while condition:`. Attention aux boucles infinies ! `import random` puis `random.randint(1, 6)` pour un de.

▶ Solution

Exercice 3.4 : Fonctions simples [Difficulté : Intermédiaire]

Énoncé :

1. Ecrivez une fonction `celsius_to_fahrenheit(c)` qui convertit Celsius en Fahrenheit (F = C x 9/5 + 32)
2. Ecrivez une fonction `imc(poids, taille)` qui calcule l'indice de masse corporelle (poids en kg, taille en m)
3. Ecrivez une fonction `est_premier(n)` qui retourne `True` si `n` est un nombre premier
4. Testez chaque fonction avec plusieurs valeurs

Indice : La syntaxe est `def nom_fonction(parametre):`. Utilisez `return` pour retourner une valeur. Un nombre premier n'est divisible que par 1 et lui-meme.

▶ Solution

Exercice 3.5 : Arguments par défaut et *args [Difficulté : Intermédiaire]

Énoncé :

1. Ecrivez une fonction `saluer(nom, salutation="Bonjour")` avec un argument par defaut
2. Ecrivez une fonction `puissance(base, exposant=2)` qui calcule base^exposant
3. Ecrivez une fonction `moyenne(*args)` qui accepte un nombre variable d'arguments et retourne leur moyenne
4. Ecrivez une fonction `fiche(**kwargs)` qui affiche toutes les paires cle-valeur passees en argument

Indice : `*args` recoit les arguments positionnels en tuple. `**kwargs` recoit les arguments nommes en dictionnaire.

▶ Solution

Exercice 3.6 : Compréhensions de liste [Difficulté : Intermédiaire]

Énoncé :

1. Creez une liste des carres des nombres de 1 a 15
2. Creez une liste des nombres pairs de 1 a 30
3. A partir de `mots = ["Python", "est", "un", "langage", "puissant"]`, creez une liste des longueurs de chaque mot
4. Filtrez les mots de plus de 3 caracteres
5. Creez une liste de tuples `(nombre, carre)` pour les nombres de 1 a 10
6. Aplatissez une liste de listes : `[[1, 2], [3, 4], [5, 6]]` en une seule liste

Indice : Syntaxe : `[expression for x in iterable]` ou `[expression for x in iterable if condition]`. Pour aplatir : `[elem for sous_liste in liste for elem in sous_liste]`.

▶ Solution

Exercice 3.7 : Fonctions avancées [Difficulté : Avancé]

Énoncé :

1. Creez une fonction lambda qui calcule le carre d'un nombre et testez-la
2. Utilisez `map()` pour convertir une liste de temperatures Celsius en Fahrenheit
3. Utilisez `filter()` pour garder uniquement les nombres positifs de `[-3, 7, -1, 4, -8, 2, 0, 5]`
4. Utilisez `sorted()` avec le parametre `key` pour trier une liste de tuples `("nom", age)` par age
5. Combinez `map()` et `filter()` : a partir d'une liste de nombres, gardez les pairs et calculez leurs carres
6. Utilisez `sorted()` avec une lambda pour trier des mots par leur derniere lettre

Indice : `lambda x: expression` cree une fonction anonyme. `map(fonction, iterable)` applique une fonction. `filter(fonction, iterable)` filtre. `sorted(iterable, key=fonction)` trie.

▶ Solution